How to make almost anything ?

a project by Alexandre COURRIEU, Théo RENOUARD, Mahel MECHERFI and Océane LAM

Goal of the project = assembling a PCB card that can modify a video based on a Rasberry PI

Here are the components that we need to solder for the card :

- **bread board**
- ***3 POTS**
- ***1 VGA**
- ***3 DIP**
- **40 pin header resistors**
- ***3 AMPS Headers**

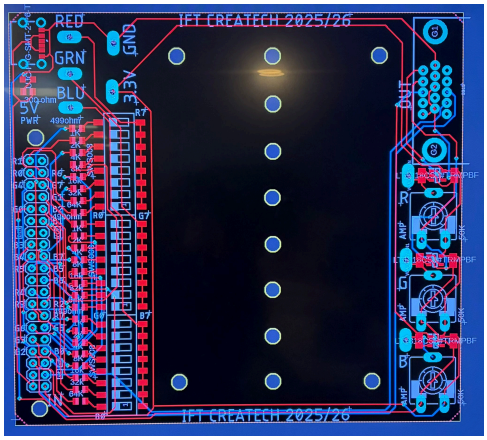
We gathered all the components on a white sheet of paper and wrote the name of each component along with its value in order to know exactly which item to use for soldering.

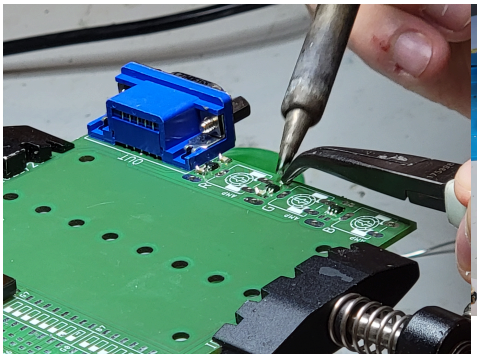
This made it easier for us to know precisely which component to use during the soldering process. In fact, this method helps to avoid most mistakes, as people can easily use resistors with different values. This could otherwise lead to common errors. Thanks to this method, the work involves not only making fewer mistakes, but also learning the names of the components, while keeping our work cleaner and more organized.

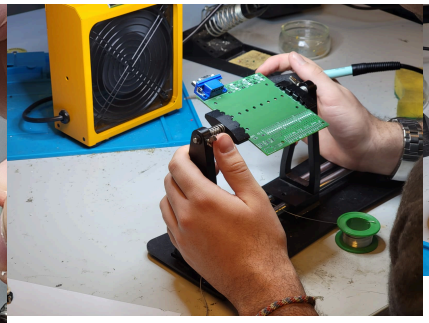
- **USB - C**
- **LED Caps**

Based on the Kicad schema that was given to us, we just had to follow it to put each component at their place :

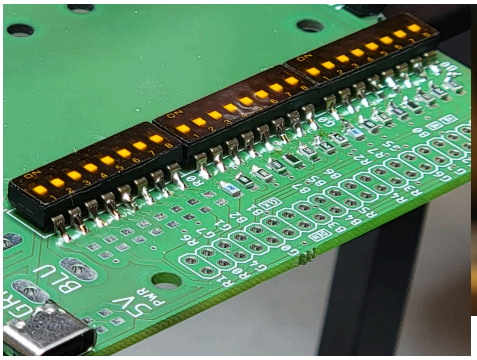
Here are some shots of our process of making the PCB card :

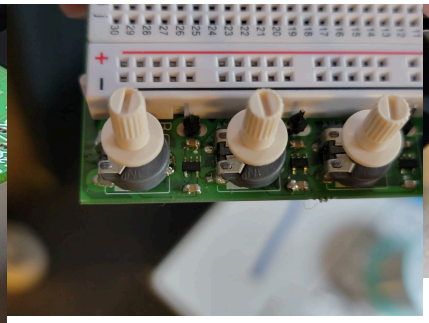


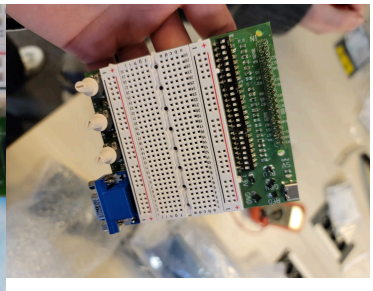












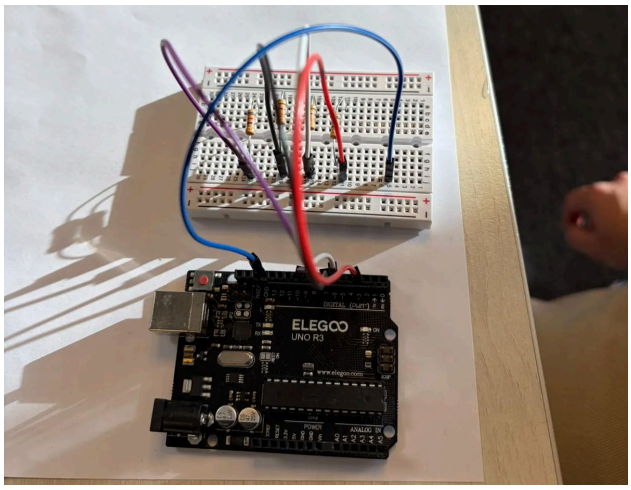
After finishing the soldering process, we had to test whether the PCB card worked correctly. Here are the first tests that we carried out.

- The **first time** we tried to check if the card worked, it did not. In fact, the LED component did not light up because we soldered the LED the wrong way.

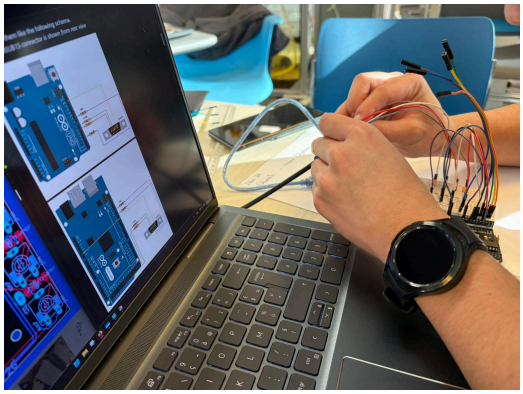
- The **second time** we tried, the card still did not work. We then realized that the problem was related to the interrupters, which were not set to the “ON” position :

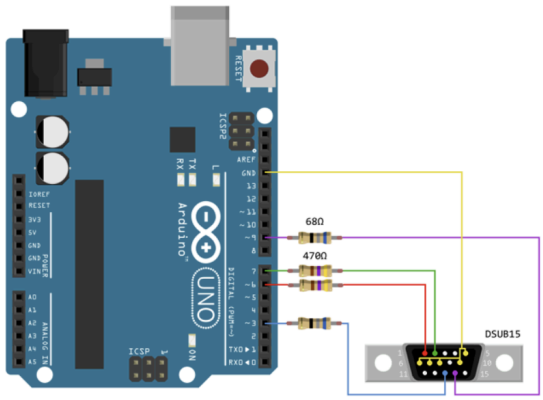


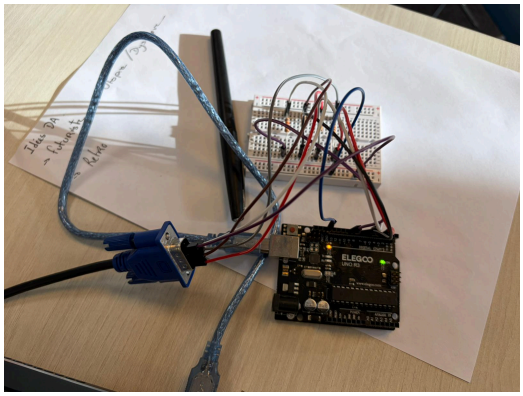
- The **third time**, the card successfully worked. But only one out of the three colors only worked : the green one. It wasn’t caused by the PCB card himself but it was because of the amplifiers on the card.



We were unable to display the video using the Raspberry Pi, so the next step was to use the Arduino Uno board. Therefore, here is our circuit based on the Arduino schematic and the **library "VGAX"**.







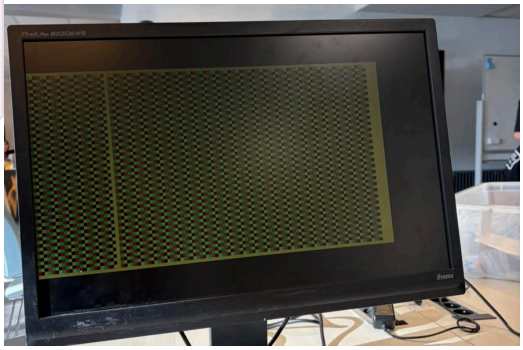
Here is the link of the github we used to get all the the full informations :

Here is what appeared on the screen after uploading the code on the card :

GitHub - smaffer/vgax: VGA library for Arduino UNO

VGA library for Arduino UNO. Contribute to smaffer/vgax development by creating an account on GitHub.

https://github.com/smaffer/vgax?tab=readme-ov-file



To test our card we put this following code :

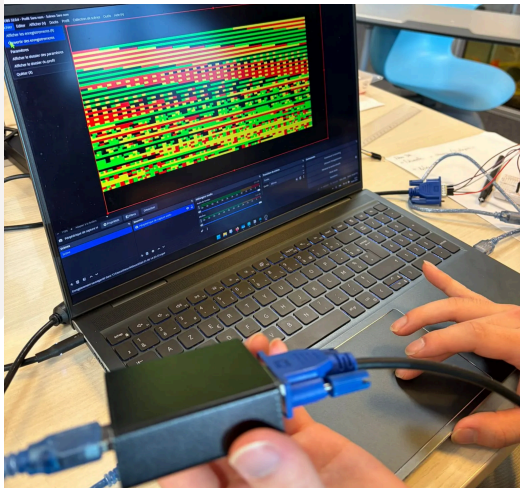
```
#include <VGAX.h>
VGAX vga;

void setup() {
vga.begin();
vga.clear(11);
for (int y=0; y!=VGAX_HEIGHT; y++)
{
for (int x=0; x!=VGAX_BWIDTH; x++)
{
if (x==0 || x==VGAX_BWIDTH-1)
vga.putpixel(x, y, 3);
else if (y==0 || y==VGAX_HEIGHT-1)
vga.putpixel4(x, y, 0xff);
else
vga.putpixel4(x, y, y%2 ? 27 : 22
8);
}
}
}
void loop() {
}
```

This is the video with patterns that we tried with a different sound we made.

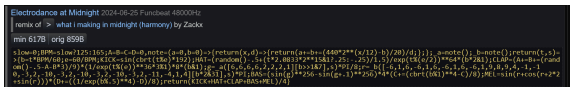
attachment:96273e7d-04e7-4a58-8b6a-76a34504deda:WhatsApp_Video_2026-01-06_at_14.24.33.mp4

Trying the card with the sound, here is what we got on the screen .





▼ byte bit



Here is the code to have the visuals of the byte bit music

```
#include <VGAX.h>
#include <avr/pgmspace.h>
// Nécessaire pour lire la
mémoire Flash

VGAX vga;
static uint32_t t = 0;

// --- DONNÉES EN MÉMOIRE
FLASH (PROGMEM) ---
// On stocke ces tableaux
ici pour sauver la RAM.
// Ils ne peuvent plus êtr
e modifiés, seulement lus.

// Table de sinus pré-calc
```

Trying with other sounds :

attachment:fc44a08c-894e-4930-be6f-03e87724985b:WhatsApp_Video_2026-01-06_at_16.58.12.mp4

https://youtu.be/TZLiMWhMefs

So here is the pattern with the sound “what I’m making at midnight”.

```
ulée (256 valeurs, -127 à
127)
const int8_t sin_tab[] PRO
GMEM = {
  0, 3, 6, 9, 12, 15, 18,
21, 24, 28, 31, 34, 37, 4
0, 43, 46, 48, 51, 54, 57,
60, 63, 65, 68, 71, 73, 7
6, 78, 81, 83, 85, 88,
  90, 92, 94, 96, 98, 100,
102, 104, 106, 107, 109, 1
11, 112, 113, 115, 116, 11
7, 118, 120, 121, 122, 12
2, 123, 124, 125, 125, 12
6, 126, 126, 127, 127, 12
7,
  127, 127, 127, 127, 126,
126, 126, 125, 125, 124, 1
23, 122, 122, 121, 120, 11
8, 117, 116, 115, 113, 11
2, 111, 109, 107, 106, 10
4, 102, 100, 98, 96, 94, 9
2,
  90, 88, 85, 83, 81, 78,
76, 73, 71, 68, 65, 63, 6
0, 57, 54, 51, 48, 46, 43,
40, 37, 34, 31, 28, 24, 2
1, 18, 15, 12, 9, 6, 3,
  0, -3, -6, -9, -12, -15,
-18, -21, -24, -28, -31, -
34, -37, -40, -43, -46, -4
8, -51, -54, -57, -60, -6
3, -65, -68, -71, -73, -7
6, -78, -81, -83, -85, -8
8,
  -90, -92, -94, -96, -98,
```

https://youtu.be/16tWUqqYGpo

attachment:af623d63-8058-4089-8421-ea9aa2c238ae:2026-01-08_14-47-29.mp4

▼ code

```
#include <VGAX.h>
#include <avr/pgmspace.h>

VGAX vga;
static uint32_t t = 0;

// Table Sinus (indispensa
ble pour les ondulations)
const int8_t sin_tab[] PRO
GMEM = {
  0, 6, 12, 18, 24, 30, 3
6, 42, 48, 54, 60, 65, 71,
76, 81, 85, 90, 94, 98, 10
2, 106, 109, 112, 115, 11
7, 120, 122, 123, 125, 12
6, 126, 127,
  127, 127, 126, 126, 125,
123, 122, 120, 117, 115, 1
12, 109, 106, 102, 98, 94,
90, 85, 81, 76, 71, 65, 6
0, 54, 48, 42, 36, 30, 24,
18, 12, 6,
  0, -6, -12, -18, -24, -3
0, -36, -42, -48, -54, -6
```

```
-100, -102, -104, -106, -1
07, -109, -111, -112, -11
3, -115, -116, -117, -118,
-120, -121, -122, -122, -1
23, -124, -125, -125, -12
6, -126, -126, -127, -127,
-127,
  -127, -127, -127, -127,
-126, -126, -126, -125, -1
25, -124, -123, -122, -12
2, -121, -120, -118, -117,
-116, -115, -113, -112, -1
11, -109, -107, -106, -10
4, -102, -100, -98, -96, -
94, -92,
  -90, -88, -85, -83, -81,
-78, -76, -73, -71, -68, -
65, -63, -60, -57, -54, -5
1, -48, -46, -43, -40, -3
7, -34, -31, -28, -24, -2
1, -18, -15, -12, -9, -6,
-3
};

const int8_t melody_seq[]
PROGMEM = { 6, 6, 6, 6, 2,
2, 2, 1 };

const int8_t bass_seq[] PR
OGMEM = {
  -6, 1, 6, -6, 1, 6, -6,
1, 6, -6, 1, 9, 8, 9, 4, -
1,
  -10, -3, 2, -10, -3, 2,
-10, -3, 2, -10, -3, 2, -1
1, -4, 1, 4
```

```
0, -65, -71, -76, -81, -8
5, -90, -94, -98, -102, -1
06, -109, -112, -115, -11
7, -120, -122, -123, -125,
-126, -126, -127,
  -127, -127, -126, -126,
-125, -123, -122, -120, -1
17, -115, -112, -109, -10
6, -102, -98, -94, -90, -8
5, -81, -76, -71, -65, -6
0, -54, -48, -42, -36, -3
0, -24, -18, -12, -6
};

void setup() {
  vga.begin();
  vga.clear(0);
}

static inline uint8_t pack
4(uint8_t c0, uint8_t c1,
uint8_t c2, uint8_t c3) {
  return (c0 << 6) | (c1 <
< 4) | (c2 << 2) | c3;
}

static inline int8_t get_s
in(uint8_t idx) {
  return (int8_t)pgm_read_
byte(&sin_tab[idx]);
}

// --- C'EST ICI QU'ON JOU
E ---
static inline uint8_t my_a
nimation(uint32_t tt, int
```

```
};

// --- FIN DES DONNÉES ---

void setup() {
  vga.begin();
  vga.clear(0);
}

static inline uint8_t pack4(uint8_t c0, uint8_t c1, uint8_t c2, uint8_t c3) {
  return (c0 << 6) | (c1 << 4) | (c2 << 2) | c3;
}

// Fonction de visualisation avec lecture PROGMEM
static inline uint8_t techno_visual(uint32_t tt, int x, int y) {

  // Vitesse globale
  uint16_t beat_time = tt >> 8;

  // Lecture des séquences depuis la mémoire FLASH via pgm_read_byte
  // melody_seq est lu tous les 8 temps (>> 3)
  int8_t m_note = (int8_t)pgm_read_byte(&melody_seq[(beat_time >> 3) & 7]);

  // bass_seq est lu tous
```

```
x, int y) {

  // IDÉE 1 : Créer du mouvement ondulant
  // On prend la position X, on ajoute le temps, et on regarde dans la table sinus
  // (tt * 2) gère la vitesse de l'onde
  int8_t wave_x = get_sin((x * 4 + tt * 3) & 255);

  // IDÉE 2 : Pareil pour Y mais avec une vitesse différente
  int8_t wave_y = get_sin((y * 8 - tt * 2) & 255);

  // IDÉE 3 : Mélanger les deux ondes pour créer le motif
  // On ajoute x et y pour que le motif soit "attaché" à l'écran et pas juste une couleur unie
  uint8_t pattern = (x + wave_x) ^ (y + wave_y);

  // IDÉE 4 : La couleur
  // On veut que ça change de couleur. On prend le motif et on le décale.
  // "(pattern >> 3)" réduit le détail pour faire des grosses zones de couleur
```

```
les temps (>> 1)
  int8_t b_note = (int8_t)
pgm_read_byte(&bass_seq[(b
eat_time >> 1) & 31]);

  // Kick (Grosse caisse)
  uint8_t kick = 0;
  if ((tt & 63) < 8) kick
= 60;

  // Calcul des ondes
  // On lit la table sinus
depuis la FLASH
  // Onde 1 (Basse)
  uint8_t wave_idx1 = (x *
2 + tt + b_note * 10) & 25
5;
  int8_t wave1 = (int8_t)p
gm_read_byte(&sin_tab[wave
_idx1]);

  // Onde 2 (Mélodie)
  uint8_t wave_idx2 = (y *
4 - tt + m_note * 20) & 25
5;
  int8_t wave2 = (int8_t)p
gm_read_byte(&sin_tab[wave
_idx2]);

  // Hat (Bruit)
  uint8_t hat = 0;
  if (tt & 32) {
     hat = ((x ^ y) * tt)
& 30;
  }
```

```
  // "+ (tt >> 4)" fait cy
cler les couleurs doucemen
t
  uint8_t color = (pattern
>> 4) + (tt >> 5);

  return color & 3; // On
garde toujours entre 0 et
3
}

void loop() {
  for (int y = 0; y < VGAX
_HEIGHT; y++) {
    for (int xb = 0; xb <
VGAX_BWIDTH; xb++) {
      int x = xb * 4;

      uint8_t c0 = my_anim
ation(t, x, y);
      uint8_t c1 = my_anim
ation(t, x+1, y);
      uint8_t c2 = my_anim
ation(t, x+2, y);
      uint8_t c3 = my_anim
ation(t, x+3, y);

      vga.putpixel4(xb, y,
pack4(c0, c1, c2, c3));
    }
  }
  t++;
}
```

```
  // Mixage
  return 128 + (wave1 / 2)
+ (wave2 / 2) + kick + ha
t;
}

void loop() {
  // Animation

  for (int y = 0; y < VGAX
_HEIGHT; y++) {
    for (int xb = 0; xb <
VGAX_BWIDTH; xb++) {
      int x = xb * 4;

      uint8_t v0 = techno_
visual(t, x, y);
      uint8_t v1 = techno_
visual(t, x+1, y);
      uint8_t v2 = techno_
visual(t, x+2, y);
      uint8_t v3 = techno_
visual(t, x+3, y);

      uint8_t c0 = v0 >>
6;
      uint8_t c1 = v1 >>
6;
      uint8_t c2 = v2 >>
6;
      uint8_t c3 = v3 >>
6;

      vga.putpixel4(xb, y,
pack4(c0, c1, c2, c3));
    }
```

```
  }

  t+=2; // Vitesse d'anima
tion (augmenter si trop le
nt)
}
```

STREAM VIDEO

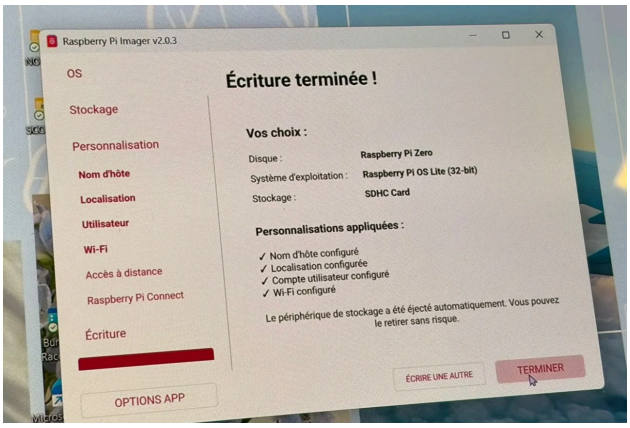
We change one more time with a raspberry PI to get a a continuous video stream.

We use a raspberry Zero this time, with the Raspberry PI OS the (32 bits), we also change the config.txt to enable the video stream by the 40 pins. **But** the rasp do not seem to work there is no video output, we change the PCB, the rasp and even the monitor, and still no video.



With the demo rasp the monitor displays the movies, the demo raps has a different OS, it is the LITE one.

We format the SD Card, to put the new OS.



We also change the config to the new one, but there is still no video, now we wait for a new SD card with the good OS in it. We need to clone the SD card.

After 3 hours of hard work a group finally made the copy of the SD card YaY.

attachment:2167fd67-a3bb-493d-9369-8c6a9ec87b01:video_hma_1.mp4

RGB switch testing

We used the RGB switches on the PCB card to modify the displayed colors.

By enabling or disabling specific switches, we were able to cut off the transmission of certain color values.

As a result, the visual output on the screen changed accordingly, allowing us to

Test with the cloned SD card

After cloning the SD card from the working demo Raspberry Pi, we tested our new PCB card using this copied system.

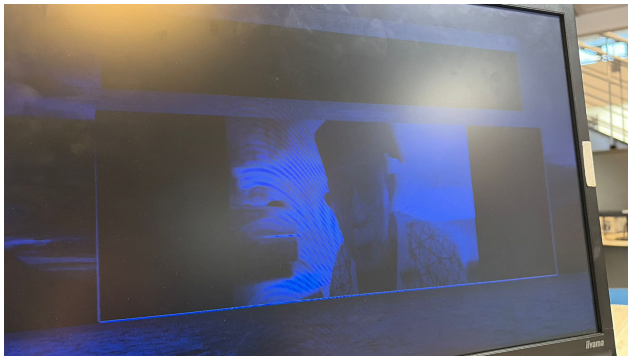
This step allowed us to ensure that the issue was not related to the operating system configuration, but rather to the hardware or the SD card itself.

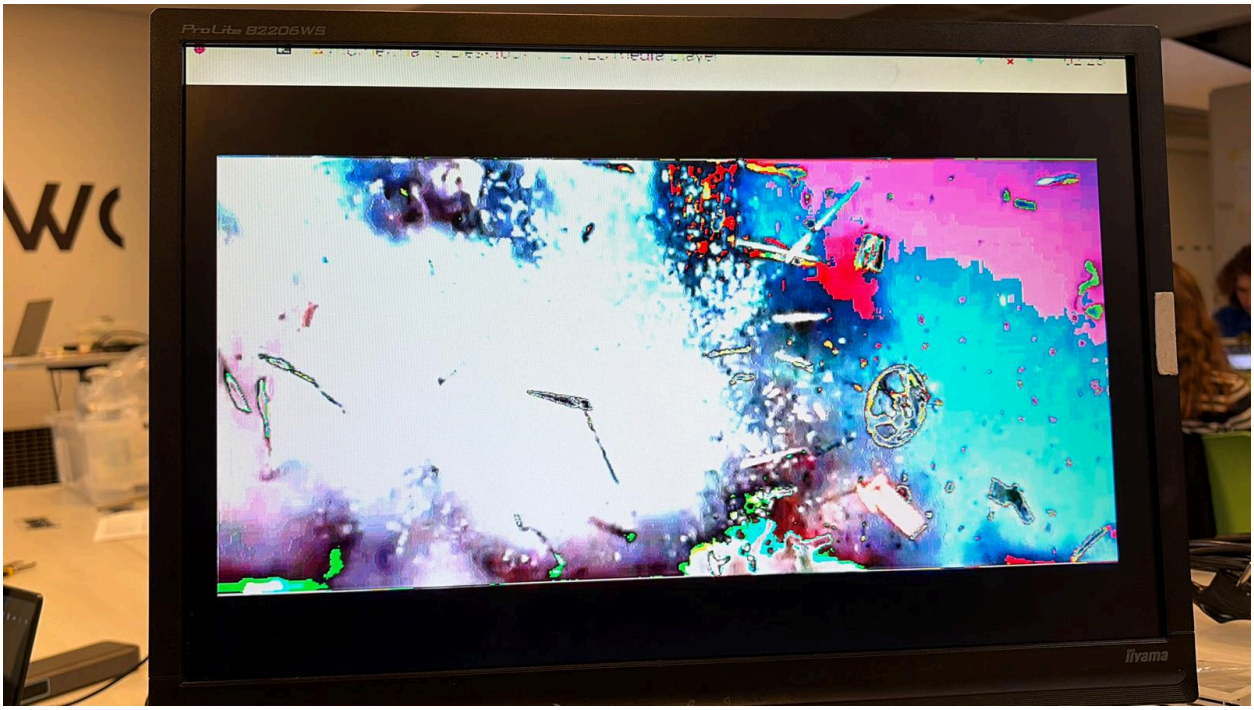
Once the cloned SD card was inserted and the setup powered on, the video signal was successfully detected by the monitor.

The video below shows what is displayed on the screen when using the cloned SD card with our PCB card.

selectively alter the color channels and observe their impact on the video rendering.

This test confirmed that the RGB switches correctly control the color signals and effectively modify the final display.Test with the cloned SD card



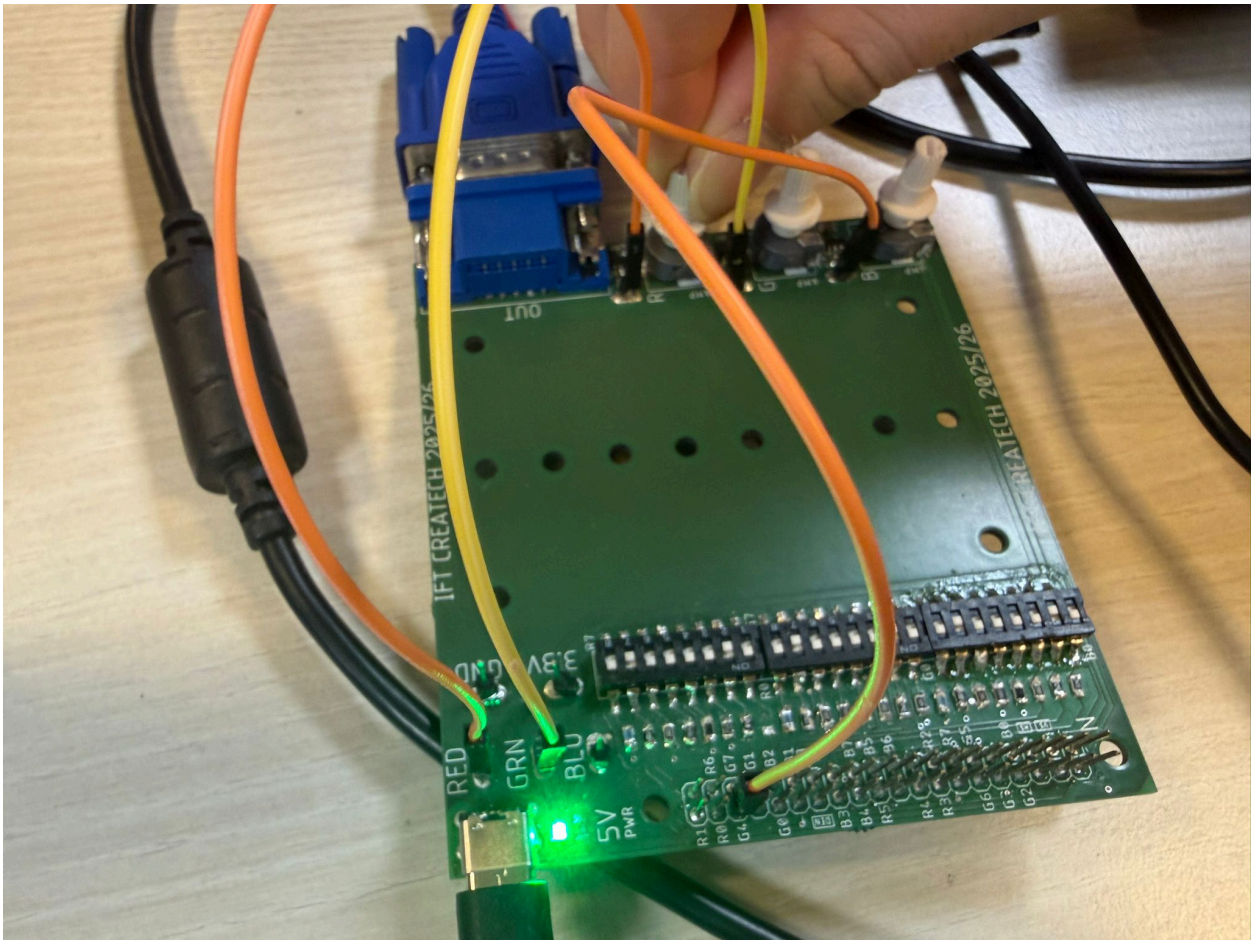


attachment:af2750fe-2a1c-428c-a344-da2e22c3c9ed:video_2_HMA.mp4

Then, we set some switches to ON and others to OFF.

By changing this combination, we modified which values were allowed to pass through the signal, which directly affected the display.

As a result, the colors changed depending on the switches that were enabled or disabled, allowing us to clearly observe the impact of each configuration on the final video output.



We continued to change the switches and this is an other result :

attachment:acc97472-c287-467b-83c0-7679f73d001f:video_3_hma.mp4

https://youtu.be/fNYfDHfh8Hk